Exp 12b: Implementation of Chat Client Server Using TCP/UDP Sockets

AIM :

To implement a multi-client chat application (chat client–server) using TCP sockets in Java, in which messages sent by one client are broadcast by the server to all the other connected clients.

ALGORITHM :

Chat Server

1. Start the experiment.
2. Create a `ServerSocket` on port 5000 and keep a thread-safe set of all connected clients.
3. Accept a client connection, create a `ClientHandler` for it, add it to the set and start a new thread to serve that client.
4. In the handler, ask the client for its name and broadcast “<name> has joined the chat.” to all the other clients.
5. Read each message from the client, display it on the server console and broadcast “<name>: <message>” to all the other clients.
6. When the client sends “/quit” or the connection is lost, remove the client from the set, broadcast “<name> has left the chat.” and close the socket.

Chat Client

1. Create a `Socket` connected to the server address (127.0.0.1) and port 5000.
2. Start a separate listener thread that continuously reads messages from the server and displays them.
3. In the main thread, read the name and messages typed by the user and send them to the server.
4. When the user types “/quit”, close the socket and stop the experiment.

CODE :

ChatServer.java

```
import java.io.*;
import java.net.*;
import java.util.*;

public class ChatServer {

    private static final int PORT = 5000;
    // Thread-safe set of all connected clients' writers, keyed by handler
    private static final Set<ClientHandler> clients = Collections.synchronizedSet(new HashSet<>());

    public static void main(String[] args) throws IOException {
        ServerSocket serverSocket = new ServerSocket(PORT);
        System.out.println("Chat server started on port " + PORT);

        while (true) {
            Socket socket = serverSocket.accept();
```

```
        ClientHandler handler = new ClientHandler(socket);
        clients.add(handler);
        new Thread(handler).start();
    }
}

// Send a message to every connected client except the sender
private static void broadcast(String message, ClientHandler sender) {
    synchronized (clients) {
        for (ClientHandler client : clients) {
            if (client != sender) {
                client.send(message);
            }
        }
    }
}

private static void removeClient(ClientHandler handler) {
    clients.remove(handler);
    broadcast(handler.getName() + " has left the chat.", handler);
    System.out.println(handler.getName() + " disconnected.");
}

// Handles a single client connection
private static class ClientHandler implements Runnable {
    private final Socket socket;
    private BufferedReader in;
    private PrintWriter out;
    private String name;

    ClientHandler(Socket socket) {
        this.socket = socket;
    }

    String getName() {
        return name;
    }

    void send(String message) {
        out.println(message);
    }

    @Override
    public void run() {
        try {
            in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            out = new PrintWriter(socket.getOutputStream(), true);

            out.println("Enter your name:");
            name = in.readLine();
            if (name == null) {
                name = "Anonymous-" + socket.getPort();
            }

            System.out.println(name + " connected from " + socket.getInetAddress());
            broadcast(name + " has joined the chat.", this);
            out.println("Welcome, " + name + "! Type /quit to exit.");

            String line;
            while ((line = in.readLine()) != null) {
                if (line.equalsIgnoreCase("/quit")) {
                    break;
                }
                System.out.println(name + ": " + line);
                broadcast(name + ": " + line, this);
            }
        } catch (IOException e) {
            System.out.println("Connection error with " + name + ": " + e.getMessage());
        } finally {
            closeConnection();
        }
    }

    private void closeConnection() {
```

```
            try {
                removeClient(this);
                socket.close();
            } catch (IOException e) {
                // ignore
            }
        }
    }
}
```

ChatClient.java

```
import java.io.*;
import java.net.*;

public class ChatClient {

    private static final String SERVER_ADDRESS = "127.0.0.1"; // change to server IP if remote
    private static final int SERVER_PORT = 5000;

    public static void main(String[] args) throws IOException {
        Socket socket = new Socket(SERVER_ADDRESS, SERVER_PORT);
        BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader console = new BufferedReader(new InputStreamReader(System.in));

        // Thread to continuously listen for messages from the server
        Thread listener = new Thread(() -> {
            String serverMessage;
            try {
                while ((serverMessage = in.readLine()) != null) {
                    System.out.println(serverMessage);
                }
            } catch (IOException e) {
                System.out.println("Disconnected from server.");
            }
        });
        listener.setDaemon(true);
        listener.start();

        // Main thread reads from console and sends to server
        String userInput;
        while ((userInput = console.readLine()) != null) {
            out.println(userInput);
            if (userInput.equalsIgnoreCase("/quit")) {
                break;
            }
        }

        socket.close();
    }
}
```

OUTPUT :

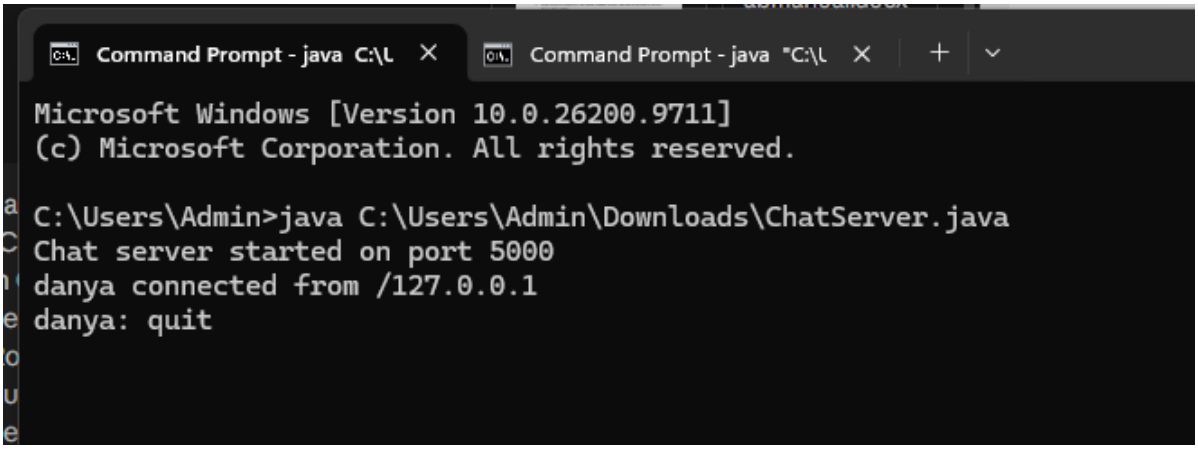


Fig 1. Chat Server – server started, client connected and its message displayed

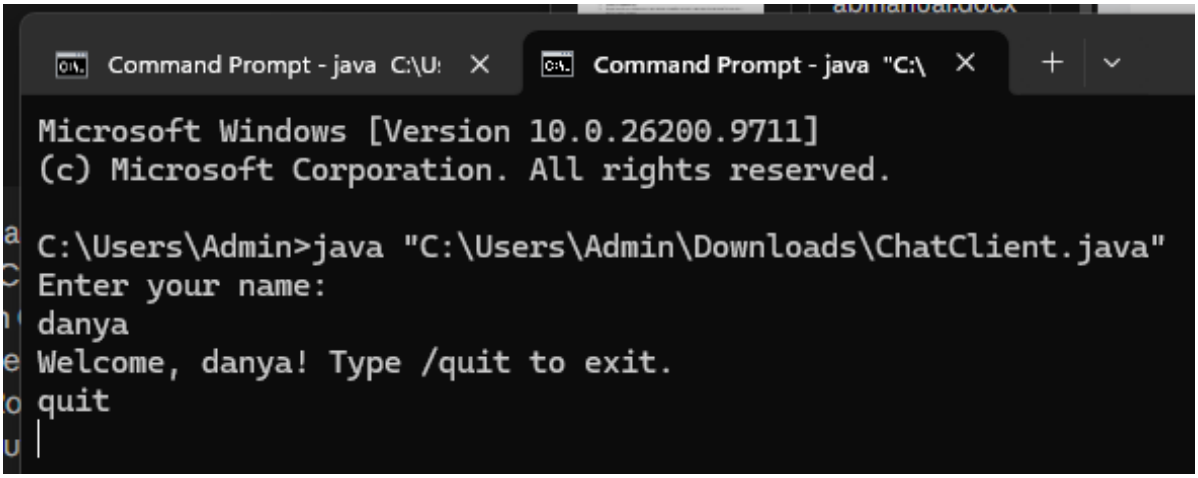


Fig 2. Chat Client – name entered, welcome message received and message sent

RESULT :

Thus, the chat client–server application was successfully implemented using TCP sockets in Java, and messages from a client were received and broadcast by the server.